

ECPC CONTEST SHEET

Built for a team targeting 3-5 solves. Recognition first, code second.

How to use this sheet

This is not a library. It is a lookup tool. Under pressure you will not read paragraphs, so everything here is a table or a snippet you can find in under ten seconds.

The section order is the order you should use them in. Sections 3 to 9 are code - only open them AFTER you know the idea. Never browse code looking for inspiration.

The full contents, with a page number for every heading, is on the next page.

The three rules that actually win problems

1. READ THE CONSTRAINTS BEFORE THE STORY. n and t decide everything.
2. 20-MINUTE RULE. No working idea after 20 minutes means the problem goes back on the pile and someone else reads it fresh. No exceptions, no ego.
3. SOLVE COUNT BEATS PROBLEM LETTER. The moment the scoreboard is visible, re-sort every problem by how many teams solved it. Problem H is often easier than problem B.

Personalise this before the contest

Two parts of this sheet are deliberately unfinished, because a sheet you built yourself is one you can navigate:

- SECTION 13 has a blank pattern index. Every problem you upsolve from now until the contest, write ONE line: the signal that gave it away, and the idea. Not the code. Sixty lines of that is worth more than any template.
- Every code block has room in the margin. When you use a snippet in practice and something bites you, write the bite next to it in pen.

Any snippet you have not personally typed and gotten Accepted with is a trap, not an asset. Test everything in this sheet at least once before contest day.

CONTENTS

Every heading in the sheet, with its page. Section titles are in bold with a line saying what the section is for; everything indented under a section is a heading you will find on that page.

1. Constraints tell you the answer	4
<i>Look at n. It tells you the complexity you are allowed, which eliminates 90% of wrong ideas before you have one. ALWAYS OPEN THIS FIRST.</i>	
The other number that matters: t	4
Integer limits - memorise these three	4
2. Trigger table - what the problem is really asking	5
<i>Match what the problem says to a technique. The most valuable pages in the sheet. Use it every single time you are stuck.</i>	
Trigger table (continued)	6
3. Templates, input, output	7
<i>Your two templates, plus the input, output and sorting syntax. Type one of these the moment you sit down.</i>	
Template A - minimal. Type this first, every time.	7
Reading input	7
Output	7
Template B - full. Use when the problem needs the helpers	8
Notes on your templates	8
Sorting and comparators	9
Searching a sorted array	9
4. The core patterns that solve the most problems	10
<i>Binary search on the answer, prefix sums, two pointers, bitmask, inversions. The patterns behind the most problems at this level.</i>	
4.1 Binary search on the answer	10
4.2 Prefix sums and the difference array	10
4.3 Two pointers / sliding window	10
4.4 Subsets with a bitmask ($n \leq 20$)	11
4.5 Fenwick tree, and counting inversions	11
5. Greedy, and what to do when you have no idea	12
<i>How to test a greedy in 60 seconds, and how to brute-force the pattern out of a problem you cannot solve. GO HERE WHEN YOU HAVE NO IDEA.</i>	
Test a greedy in 60 seconds: the exchange argument	12
The greedy shapes that keep coming back	12
Interval scheduling - the one people get wrong	12
How to kill a wrong greedy, fast	12
When you have NO idea: brute force the small cases	13
6. Number theory and combinatorics	14
<i>gcd, lcm, fast power, modular inverse, sieve, factorisation, nCr, and the formulas worth having on paper.</i>	
gcd, lcm, fast power, modular inverse	14
Sieve of Eratosthenes	14
Factorisation and divisors	14
nCr modulo a prime	14
Formulas worth having on paper	15
7. Graphs	16
<i>DSU, BFS, grid BFS, 0-1 BFS, Dijkstra, Floyd-Warshall, DFS, components, bipartite check.</i>	
DSU (union-find) - use it far more than you think	16
BFS on a graph - shortest path when every edge costs 1	16
BFS on a grid	16
Dijkstra - shortest path with positive weights	16
0-1 BFS - when every edge costs 0 or 1	17
Floyd-Warshall - all pairs, $n \leq 400-500$	17
DFS, connected components, bipartite check	17
8. The dynamic programming you will actually need	18
<i>The six recurrences that cover almost every DP you will meet at 1300-1600.</i>	
0/1 knapsack - each item at most once	18
Coin change - unlimited copies	18
Longest increasing subsequence, $O(n \log n)$	18
Edit distance and LCS - the classic $O(n*m)$ grid	18
Paths in a grid	18
Linear DP on the array itself	19
9. STL and strings - stop wasting minutes on syntax	20
<i>map, set, multiset, coordinate compression, strings, and the small syntax you should never lose a minute to.</i>	
map, set, counting	20
Coordinate compression - big values, small n	20
Strings	20

Small things that cost people real minutes	20
10. Read this before every single submission	21
<i>The overflow, multi-test, edge-case and output checks. Read it before EVERY submission. Cheaper than a penalty.</i>	
The overflow check	21
The multi-test-case check	21
The edge-case check - run these in your head	21
The output check	21
11. Red verdict? Go here before touching the code	22
<i>WA, TLE, RE, MLE. Go here before you touch the code.</i>	
WRONG ANSWER	22
TIME LIMIT EXCEEDED	22
RUNTIME ERROR	22
MEMORY LIMIT EXCEEDED	22
12. Team protocol - where the extra solve comes from	23
<i>The minute-by-minute timeline and the hard rules. Read once before the contest starts, then obey it.</i>	
The timeline	23
Hard rules	23
First three minutes on any new problem	23
13. Your pattern index - fill this in before the contest	24
<i>Half-empty on purpose. You fill it in during practice - it is the part of this sheet that becomes yours.</i>	
Starter entries - recurring ideas at the 1300-1600 level	24
Your own entries - write these in pen as you practise	25
Your own entries (continued)	26

1. Constraints tell you the answer

Roughly 100,000,000 (1e8) simple operations per second. A 1-second limit means your total work must be under about 1e8. Read n , then pick from this table. If nothing fits, your idea is wrong - do not start typing.

Largest n you see	Complexity you are allowed, and what that means
$n \leq 11$	$O(n!)$ - try every permutation. <code>next_permutation</code> .
$n \leq 20$ (or ≤ 24)	$O(2^n)$ - loop over every subset with a bitmask. Very strong signal.
$n \leq 40$	$O(2^{n/2})$ - meet in the middle. Rare at this level; usually means you misread n .
$n \leq 100$	$O(n^3)$ - triple loop, Floyd-Warshall, interval DP.
$n \leq 500$	$O(n^3)$ is borderline; $O(n^2 \log n)$ is safe.
$n \leq 2,000 - 5,000$	$O(n^2)$ - two nested loops, 2D DP over pairs, LCS, edit distance.
$n \leq 100,000$ (1e5)	$O(n \log n)$ - sorting, set/map, binary search, prefix sums, one DSU pass.
$n \leq 1,000,000$ (1e6)	$O(n)$ or $O(n \log \log n)$ - single pass, sieve. No map, no sort inside a loop.
n up to 1e9	$O(\sqrt{n})$ or $O(\log n)$ - factorisation by trial division, binary search, or a formula.
n up to 1e18	$O(\log n)$ or $O(1)$ - fast power, math formula, digit DP. NEVER loop to n .

The other number that matters: t

Multi-test problems almost always say "the sum of n over all test cases does not exceed $2e5$ ". That sentence is a promise and a trap:

- The promise: your per-test work may be $O(n \log n)$ even though t is 10,000.
- The trap: you may NOT clear a global array of size $1e6$ once per test case. That is $1e10$ operations. Size your containers to n , not to N .
- If the sum of n is NOT bounded, then $t \times n$ is the real input size. Check it.

Integer limits - memorise these three

Type	Range / note
int	about $-2.1e9$ to $+2.1e9$. Anything that can reach 2 billion must not be an int.
long long	about $-9.2e18$ to $+9.2e18$. Default to this when in doubt.
$1 \ll k$	This is an INT. For $k \geq 31$ you must write <code>1LL << k</code> .

2. Trigger table - what the problem is really asking

Left column: something you can see in the statement. Right column: what it is probably about. This is a recognition aid, not a proof. Scan it whenever you are stuck for more than five minutes.

Structure and size

If the statement says / shows...	Think...
$n \leq 20-24$	Bitmask over all subsets. Loop mask from 0 to $(1 \ll n) - 1$.
$n \leq 10$	Try all permutations, or full recursive brute force.
Answer is asked modulo $1e9+7$	It is a COUNTING problem. Counting DP, or combinatorics with nCr .
Values up to $1e9$ but n only $1e5$	The VALUES are not the state. Sort, compress coordinates, or binary search on value.
"print any valid answer"	Constructive. Find one greedy that works; do not optimise. Test $n=1$ and $n=2$ by hand.
"it is guaranteed that a solution exists"	Constructive again. Look for the trivially symmetric construction.

Optimisation words

If the statement says / shows...	Think...
"minimise the maximum" / "maximise the minimum"	BINARY SEARCH ON THE ANSWER. Write a <code>check(x)</code> that returns can-we-do-it-with-x. This is the single most common 1400-1600 idea.
"the smallest T such that ..." and bigger T is always easier	Binary search on the answer. Verify monotonicity out loud before coding.
"minimum number of operations"	Greedy sweep left to right, or count what is forced. Ask: what does each operation cost, and what is the lower bound?
"minimum cost to make all elements equal", cost = $ a[i] - x $	The optimum x is the MEDIAN. Sort, take $a[n/2]$.
"...", cost = $(a[i] - x)^2$	The optimum x is the MEAN. Check both floor and ceil.
Operations may only INCREASE values (or only decrease)	Sweep once. <code>answer += max(0, needed - current)</code> . No search required.
"choose k of them to maximise the sum"	Sort descending, take the top k. If there is a constraint linking them, sort by the constraint.
Two arrays, pair them up to maximise/minimise total	Sort both. Same order = maximise. Opposite order = minimise.

Arrays, subarrays, queries

If the statement says / shows...	Think...
Range sum queries, array never changes	Prefix sums. $O(1)$ per query.
"add v to every element from l to r ", many times, print at the end	DIFFERENCE ARRAY. <code>d[l] += v</code> , <code>d[r+1] -= v</code> , then one prefix pass.
Sum / count over ALL subarrays	Do not enumerate them. Count each element's CONTRIBUTION: how many subarrays contain index i .
"longest subarray such that <condition>"	Sliding window / two pointers - IF extending the window can only make the condition worse.
Sorted array, find a pair summing to X	Two pointers from both ends, or <code>lower_bound</code> for each element.
Count pairs (i, j) with $i < j$ and <condition>	Sweep j left to right, and query a map/count-array holding all $i < j$.
Rectangle sums on a grid	2D prefix sums.
"you may change at most one element"	Precompute prefix and suffix answers, then try all n positions in $O(n)$ total.
"distinct" / "how many different"	set or map. Or sort then count adjacent-unequal.
k -th smallest	Sort. If the array is huge or virtual, binary search on the VALUE.

Trigger table (continued)

Graphs

If the statement says / shows...	Think...
"connect", "merge", "are these two in the same group"	DSU (union-find). 15 lines, always worth it.
"number of connected components", "is it all connected"	DSU, or BFS/DFS from every unvisited node.
Shortest path, every edge costs 1 (or is a grid)	BFS. Not Dijkstra.
Shortest path, positive weights	Dijkstra with a priority_queue.
n nodes and exactly n-1 edges, connected	It is a TREE. No cycles, exactly one path between any two nodes, root it anywhere.
"split into two teams so that no ..."	Bipartite check: BFS colouring with two colours. A conflict means odd cycle, answer NO.
$n \leq 400$ -500 and all-pairs distances	Floyd-Warshall, triple loop, k outermost.
"can you reach X from Y"	BFS/DFS reachability, or DSU if edges are undirected and static.
Grid with walls, move in 4 directions	Grid BFS with the dx/dy arrays. Guard the bounds FIRST, then the wall check.

Numbers

If the statement says / shows...	Think...
gcd, lcm, "divisible by"	Factorise, or use __gcd. $\text{lcm} = a / \text{gcd} * b$ (divide first, or you overflow).
"how many divisors", "list the divisors"	Loop i from 1 while $i*i \leq n$. Each hit gives you i and n/i .
Primes up to $1e6$ or so	Sieve once at the start of main, not per test case.
"is n prime", n up to $1e12$	Trial division up to $\sqrt{n}$. That is only $1e6$ steps.
XOR anywhere in the statement	Think bit by bit. The 30 bits are independent. $x \oplus x = 0$, $x \oplus 0 = x$.
"number of ways to choose k out of n"	nCr with precomputed factorials and modular inverse.
Huge exponent, answer mod p	Fast power (binary exponentiation).
Sum $1 + 2 + \dots + n$	$n * (n + 1) / 2$. Use long long. This overflows int at n around 65,000.

Strings and games

If the statement says / shows...	Think...
Anagram / rearrangement	Sort both strings and compare, or count the 26 letters.
Palindrome	Two pointers from both ends. For "can it be rearranged into one": at most one odd letter count.
Only lowercase letters	A 26-length count array is almost always the intended state.
Two players, both play optimally, who wins	At this rating it is nearly always PARITY or a tiny pattern. Brute force $n = 1..8$ by hand, find the pattern, print it.
"the first player to be unable to move loses"	Same as above. Find the losing positions for small n.

When nothing on this page matches

Do these four things, in order, on paper:

1. Solve $n = 1$, $n = 2$, $n = 3$ completely by hand. Write the answers down.
2. Look at the SAMPLE EXPLANATION, not just the sample output. It usually gives away the intended idea.
3. Ask: is the answer just the sum? the max? max minus min? the count of something? the parity? It is embarrassing how often it is.
4. Explain the problem out loud to a teammate in one sentence. If you cannot, you have misread it - go back and re-read the statement.

3. Templates, input, output

Template A - minimal. Type this first, every time.

Under 20 lines. Get this down in under a minute, then think. Delete the two t lines if the problem has a single test case.

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

void fast_io() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}

void solve() {
    // Write solution here
}

int main() {
    fast_io();

    int t;
    cin >> t;
    while (t--) {
        solve();
    }

    return 0;
}
```

Reading input

```
int n; cin >> n;
vector<ll> a(n);
for (int i = 0; i < n; i++) cin >> a[i];

// grid
int n, m; cin >> n >> m;
vector<string> g(n);
for (int i = 0; i < n; i++) cin >> g[i];    // g[i][j] is a char

// graph, 1-indexed, m edges
vector<int> adj[200005];
for (int i = 0; i < m; i++) {
    int u, v; cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);    // DELETE this line if the graph is DIRECTED
}
```

Output

```
cout << ans << "\n";    // "\n", never endl (endl flushes = slow)
cout << (ok ? "YES" : "NO") << "\n";    // copy the EXACT case from the statement
for (ll x : a) cout << x << " ";    // print a whole array
cout << "\n";
cout << fixed << setprecision(6) << x;    // floating point answers
```

Template B - full. Use when the problem needs the helpers.

Note the difference at the bottom: `t` is 1 and the `cin >> t` line is COMMENTED OUT. Uncomment it for multi-test problems. Getting this backwards is a guaranteed wrong answer on test 1.

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using ull = unsigned long long;
using ld = long double;

using pii = pair<int, int>;
using pll = pair<ll, ll>;

template <typename T>
using vec = vector<T>;

#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (int)(x).size()

const int INF = (int)1e9;
const ll LINF = (ll)4e18;
const int MOD = 1'000'000'007;

void fast_io() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}

ll gcdll(ll a, ll b) { return b ? gcdll(b, a % b) : a; }
ll lcmll(ll a, ll b) { return a / gcdll(a, b) * b; }

ll mod_pow(ll a, ll e, ll mod = MOD) {
    a %= mod;
    ll result = 1;
    while (e > 0) {
        if (e & 1) result = (result * a) % mod;
        a = (a * a) % mod;
        e >>= 1;
    }
    return result;
}

void solve() {
    // Write solution for one test case here.
}

int main() {
    fast_io();

    int t = 1;
    // cin >> t;
    while (t--) {
        solve();
    }

    return 0;
}
```

Notes on your templates

- `INF` is an `int` ($1e9$). Adding two `INF`s overflows. For distances and DP use `LINF`, and remember `LINF + anything` also wraps - compare before you add.
- `sz(x)` returns an `int`, so `sz(a) - 1` is safe when `a` is empty. Writing `a.size() - 1` on an empty vector gives you 18446744073709551615.
- `mod_pow` doubles as your modular inverse when `MOD` is prime: `inv(a) = mod_pow(a, MOD - 2)`.
- `lcmll` divides before multiplying, so it will not overflow. Never write `a * b / gcd`.
- **IMPORTANT:** every code snippet in the rest of this sheet uses `all(a)`. That macro exists in Template B only. If you started from Template A, either add the `#define all(x)` line or write `a.begin()`, `a.end()` out in full.

Sorting and comparators

```

sort(all(a)); // ascending
sort(all(a), greater<ll>()); // descending
reverse(all(a));

// vector<pair<int,int>> p; sort by first ascending, tie -> second descending
sort(all(p), [](pair<int,int> x, pair<int,int> y) {
    if (x.first != y.first) return x.first < y.first;
    return x.second > y.second;
});

// sort indices 0..n-1 by their value in a
vector<int> id(n);
for (int i = 0; i < n; i++) id[i] = i;
sort(all(id), [&](int x, int y) { return a[x] < a[y]; });

```

Searching a sorted array

```

// a MUST already be sorted
int i = lower_bound(all(a), x) - a.begin(); // first index with a[i] >= x
int j = upper_bound(all(a), x) - a.begin(); // first index with a[i] > x
int howManyX = j - i;
// i == a.size() means every element is < x

```

4. The core patterns that solve the most problems

4.1 Binary search on the answer

Use when the problem says minimise the maximum, maximise the minimum, or asks for the smallest T that works. Requirement: if x works then every bigger x works (or the mirror image). Say that sentence out loud before you code.

```
bool ok(ll x) {
    // return true if x is achievable / good enough
}

ll lo = 0, hi = 2e18, ans = -1;
while (lo <= hi) {
    ll mid = lo + (hi - lo) / 2;          // NOT (lo+hi)/2 -> that overflows
    if (ok(mid)) { ans = mid; hi = mid - 1; } // SMALLEST good x
    else { lo = mid + 1; }
}

// For the LARGEST good x, swap the two branches:
// if (ok(mid)) { ans = mid; lo = mid + 1; } else { hi = mid - 1; }
```

Set hi to a value that is definitely achievable, not to a clever tight bound. A loose hi costs you 3 extra iterations; a wrong hi costs you the problem.

4.2 Prefix sums and the difference array

```
// PREFIX SUMS - range sum queries, array never changes
vector<ll> pre(n + 1, 0);
for (int i = 0; i < n; i++) pre[i + 1] = pre[i] + a[i];
// sum of a[l..r] inclusive, 0-indexed:
ll s = pre[r + 1] - pre[l];

// DIFFERENCE ARRAY - many range updates, then read the final array
vector<ll> d(n + 2, 0);
// "add v to a[l..r]" becomes:
d[l] += v; d[r + 1] -= v;
// after all updates, rebuild:
ll cur = 0;
for (int i = 0; i < n; i++) { cur += d[i]; a[i] = cur; }

// 2D PREFIX SUMS - p is (n+1) x (m+1) of zeros, grid g is 0-indexed
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        p[i][j] = g[i-1][j-1] + p[i-1][j] + p[i][j-1] - p[i-1][j-1];
// sum of rows r1..r2, cols c1..c2 (all 1-indexed):
ll s2 = p[r2][c2] - p[r1-1][c2] - p[r2][c1-1] + p[r1-1][c1-1];
```

4.3 Two pointers / sliding window

Use when you want the longest (or shortest) window satisfying a condition, AND growing the window can only make the condition harder to satisfy.

```
// longest subarray with sum <= K (requires all a[i] >= 0)
ll sum = 0; int l = 0, best = 0;
for (int r = 0; r < n; r++) {
    sum += a[r];
    while (sum > K) sum -= a[l++]; // shrink from the left until valid
    best = max(best, r - l + 1);
}

// longest window with at most K distinct values
map<ll,int> cnt; int l = 0, best = 0;
for (int r = 0; r < n; r++) {
    cnt[a[r]]++;
    while ((int)cnt.size() > K) {
        if (--cnt[a[l]] == 0) cnt.erase(a[l]);
        l++;
    }
    best = max(best, r - l + 1);
}
```

4.4 Subsets with a bitmask ($n \leq 20$)

```
for (int mask = 0; mask < (1 << n); mask++) {
    ll sum = 0;
    for (int i = 0; i < n; i++)
        if (mask & (1 << i)) sum += a[i];    // item i is IN this subset
    // evaluate sum
}

__builtin_popcount(mask)    // number of set bits, for int
__builtin_popcountll(mask) // for long long
mask & (1 << i)             // is bit i set
mask | (1 << i)             // set bit i
mask ^ (1 << i)             // flip bit i

// all permutations instead (n <= 8-10):
sort(all(a));
do {
    // check this ordering
} while (next_permutation(all(a)));
```

4.5 Fenwick tree, and counting inversions

The only data structure in this sheet. It earns its page because it answers one question - "how many earlier elements are smaller than this one" - which is the trigger-table row "count pairs (i, j) with $i < j$ and a condition", and it gives you inversions for free.

```
const int N = 200005;
ll bit[N];    // 1-INDEXED. upd(0,..) loops forever.

void upd(int i, ll v) { for (; i < N; i += i & -i) bit[i] += v; }
ll qry(int i) { ll s = 0; for (; i > 0; i -= i & -i) s += bit[i]; return s; }
// qry(i) = sum of positions 1..i
// range = qry(r) - qry(l - 1)

// COUNTING INVERSIONS = minimum number of ADJACENT SWAPS to sort the array
// First compress a[] into 1..n (see section 9), then sweep right to left:
ll inv = 0;
for (int i = n - 1; i >= 0; i--) {
    inv += qry(a[i] - 1);    // later elements strictly smaller
    upd(a[i], 1);
}
```

5. Greedy, and what to do when you have no idea

Greedy is the most common thing you will meet at this level and the only one where the code is trivial and the thinking is everything. The question is never "how do I write it" - it is "is it right".

Test a greedy in 60 seconds: the exchange argument

Your greedy processes things in some order. Ask one question: if I take two items that are ADJACENT in that order and swap them, can the answer ever get better? If the answer is never, your order is optimal. That is a complete proof and it fits on the back of the statement.

Do this on paper before you type. A greedy you have checked is worth more than two greedies you have guessed.

The greedy shapes that keep coming back

Shape	When you reach for it
Sort ascending, sweep once, fix each violation as you meet it	"minimum operations to make the array satisfy X". The fix at position i is forced, so there is nothing to choose.
Sort descending, take the first k	"choose k of them to maximise the sum". With prefix sums you answer every k at once.
Sort by END time, take an interval if it still fits	Maximum number of non-overlapping intervals / meetings. Sorting by START is the classic wrong answer.
Two pointers from both ends	"pair them all up". Pair smallest with largest to balance; pair neighbours to minimise total difference.
Sort both arrays the same way	Maximise the sum of products. Sort them oppositely to minimise it.
Take the median	Cost is the sum of $ a[i] - x $. Take the mean instead when the cost is squared.
Do the forced moves first	Some elements have only one legal choice. Apply those, then re-look - the rest is usually easy.
Always repair the currently cheapest violation	"minimum total cost to fix everything", when repairs do not interact.

Interval scheduling - the one people get wrong

```
// maximum number of non-overlapping intervals
sort(all(v), [](pii x, pii y) { return x.second < y.second; }); // by END
int taken = 0, lastEnd = -1e9;
for (auto [s, e] : v)
    if (s >= lastEnd) { taken++; lastEnd = e; }
```

How to kill a wrong greedy, fast

- Build $n = 3$ with one extreme value: one huge, one zero, one negative. Most wrong greedies die here.
- Build the trap case on purpose: an input where taking the locally best thing first forces a terrible second step.
- Try all-equal, and try already-sorted. Your answer should be 0 operations on the already-correct input.
- If your greedy needs more than one "and also" rule, it is probably wrong. Real greedies are one sentence.
- If you cannot find a counterexample in three minutes, submit it. You are not going to prove it, and the clock is the real opponent.

When you have NO idea: brute force the small cases

This is the most underrated tool you own. It needs zero algorithmic knowledge, and it cracks the two problem types that appear constantly at 1300-1600: games with optimal play, and "print any valid construction". Run it on your own machine, read the output, write down the formula, then delete it.

Any problem: print the answer for $n = 1, 2, 3, \dots$

```
// Print the answer for every small n, then LOOK at the list of answers.
for (int n = 1; n <= 12; n++) {
    ll best = -1;
    for (int mask = 0; mask < (1 << n); mask++) {
        // build the candidate described by mask, evaluate it, update best
    }
    cout << n << " -> " << best << "\n";
}
```

Two-player games: compute who wins for every small n

```
// win[n] = true if the player TO MOVE wins with n items left.
// A position is winning if ANY legal move hands the opponent a losing one.
vector<bool> win(N + 1, false);
for (int n = 1; n <= N; n++)
    for (int take : moves) // the legal moves
        if (n - take >= 0 && !win[n - take]) win[n] = true;

for (int n = 1; n <= 20; n++) cout << n << " " << win[n] << "\n";
// Now stare at it. The pattern is almost always "n % k == 0" or parity.
```

What to do with the output

Write the first 15 answers in a row on paper. Then ask, in this order:

1. Is it always the same? Is it n , or $n-1$, or $n/2$, or $n*(n+1)/2$?
2. Does it depend only on n being odd or even?
3. Does it repeat with some period k ? Then the answer is about $n \% k$.
4. Does it change only at powers of 2? Then it is about bits.

If none of those fit after five minutes, the problem is not a pattern problem. Put it down and read a different one.

6. Number theory and combinatorics

gcd, lcm, fast power, modular inverse

```

ll g = __gcd(a, b); // built in, works on long long
ll l = a / __gcd(a, b) * b; // DIVIDE FIRST or a*b overflows

ll pw(ll b, ll e, ll m) { // b^e mod m, in O(log e)
    ll r = 1; b %= m;
    while (e > 0) {
        if (e & 1) r = r * b % m;
        b = b * b % m;
        e >>= 1;
    }
    return r;
}

// modular inverse, valid only when m is PRIME:
ll inv = pw(a, MOD - 2, MOD);
// division under a modulus: (x / y) mod p == x * pw(y, p-2, p) % p

```

Sieve of Eratosthenes

```

const int N = 1000006;
vector<bool> comp(N, false);
vector<int> primes;
void sieve() {
    for (int i = 2; i < N; i++) {
        if (comp[i]) continue;
        primes.push_back(i);
        for (ll j = (ll)i * i; j < N; j += i) comp[j] = true;
    }
}
// call sieve() ONCE at the top of main, never inside solve()

```

Factorisation and divisors

```

// prime factorisation of n, O(sqrt n). Works up to about 1e12 comfortably.
vector<pair<ll,int>> f;
ll m = n;
for (ll p = 2; p * p <= m; p++) {
    if (m % p) continue;
    int c = 0;
    while (m % p == 0) { m /= p; c++; }
    f.push_back({p, c});
}
if (m > 1) f.push_back({m, 1}); // the leftover is prime - do not forget it

// ALL divisors of n, O(sqrt n)
vector<ll> divs;
for (ll i = 1; i * i <= n; i++) {
    if (n % i) continue;
    divs.push_back(i);
    if (i != n / i) divs.push_back(n / i);
}
sort(all(divs));

```

nCr modulo a prime

```

const int MX = 200005;
ll f[MX], inv_f[MX];

void initC() {
    f[0] = 1;
    for (int i = 1; i < MX; i++) f[i] = f[i-1] * i % MOD;
    inv_f[MX-1] = pw(f[MX-1], MOD - 2, MOD);
    for (int i = MX - 1; i > 0; i--) inv_f[i-1] = inv_f[i] * i % MOD;
}

ll nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return f[n] * inv_f[r] % MOD * inv_f[n-r] % MOD;
}
// call initC() once in main

```

Formulas worth having on paper

Thing	Formula (use long long)
$1 + 2 + \dots + n$	$n * (n + 1) / 2$
Number of pairs from n items	$n * (n - 1) / 2$
Sum of an arithmetic series	$\text{count} * (\text{first} + \text{last}) / 2$
Number of subsets of n items	2^n
Ceiling division of a by b ($a, b > 0$)	$(a + b - 1) / b$
Careful: integer division truncates toward zero	$-7 / 2$ is -3 in C++, not -4
Digits of n in base 10	<code>to_string(n).size()</code> , or loop $n /= 10$

7. Graphs

DSU (union-find) - use it far more than you think

```
const int N = 200005;
int par[N], sz[N];

void initDSU(int n) {
    for (int i = 0; i <= n; i++) { par[i] = i; sz[i] = 1; }
}
int find(int x) {
    return par[x] == x ? x : par[x] = find(par[x]);
}
bool unite(int a, int b) { // returns false if already together
    a = find(a); b = find(b);
    if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b);
    par[b] = a;
    sz[a] += sz[b];
    return true;
}
// number of components = count of i in 1..n with find(i) == i
// size of i's component = sz[find(i)]
```

BFS on a graph - shortest path when every edge costs 1

```
vector<int> dist(n + 1, -1);
queue<int> q;
dist[s] = 0; q.push(s);
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u])
        if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
}
// dist[v] == -1 means v is unreachable
```

BFS on a grid

```
int dx[4] = {0, 0, 1, -1};
int dy[4] = {1, -1, 0, 0};
// 8 directions: dx[8]={0,0,1,-1,1,1,-1,-1}, dy[8]={1,-1,0,0,1,-1,1,-1};

vector<vector<int>>> d(n, vector<int>(m, -1));
queue<pair<int,int>> q;
d[sr][sc] = 0; q.push({sr, sc});
while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (int k = 0; k < 4; k++) {
        int nr = r + dx[k], nc = c + dy[k];
        if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue; // BOUNDS FIRST
        if (g[nr][nc] == '#' || d[nr][nc] != -1) continue;
        d[nr][nc] = d[r][c] + 1;
        q.push({nr, nc});
    }
}
```

Dijkstra - shortest path with positive weights

```
vector<pair<int,ll>> adj[N]; // adj[u] = list of {v, weight}
vector<ll> dist(n + 1, INF);
priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>> pq;

dist[s] = 0; pq.push({0, s});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue; // stale entry, skip
    for (auto [v, w] : adj[u])
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            pq.push({dist[v], v});
        }
}
```

0-1 BFS - when every edge costs 0 or 1

Same job as Dijkstra but $O(n + m)$ and shorter. A deque replaces the priority queue: cost-0 edges go on the FRONT, cost-1 edges on the back.

```
deque<int> dq;
vector<ll> dist(n + 1, INF);
dist[s] = 0; dq.push_back(s);
while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, w] : adj[u]) // w is 0 or 1 only
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            if (w == 0) dq.push_front(v);
            else dq.push_back(v);
        }
}
```

Floyd-Warshall - all pairs, $n \leq 400-500$

```
// d[i][j] = weight of edge i->j, INF if there is none, and d[i][i] = 0
for (int k = 1; k <= n; k++) // k MUST be the OUTER loop
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (d[i][k] != INF && d[k][j] != INF && // or INF+INF overflows
                d[i][k] + d[k][j] < d[i][j])
                d[i][j] = d[i][k] + d[k][j];
```

DFS, connected components, bipartite check

```
// DFS. Warning: a path of 1e5+ nodes can blow the stack. If n is large, use BFS.
vector<bool> vis(n + 1, false);
void dfs(int u) {
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) dfs(v);
}

// count components
int comp = 0;
for (int i = 1; i <= n; i++) if (!vis[i]) { comp++; dfs(i); }

// BIPARTITE / two-colouring: colour is -1, 0 or 1
vector<int> col(n + 1, -1);
bool bip = true;
for (int s = 1; s <= n; s++) {
    if (col[s] != -1) continue;
    queue<int> q; col[s] = 0; q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (col[v] == -1) { col[v] = 1 - col[u]; q.push(v); }
            else if (col[v] == col[u]) bip = false;
        }
    }
}
```

8. The dynamic programming you will actually need

At 1300-1600 the DP is almost never exotic. It is one of these six, lightly disguised. Identify the STATE first (what do I need to know to continue?), then the transition.

0/1 knapsack - each item at most once

```
// n items, item i has weight w[i] and value v[i], capacity W
vector<ll> dp(W + 1, 0);
for (int i = 0; i < n; i++)
    for (int c = W; c >= w[i]; c--) // BACKWARD loop = each item once
        dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
// answer = dp[W]

// SUBSET SUM (can I hit exactly S?) is the same shape with bools:
vector<bool> can(S + 1, false); can[0] = true;
for (int i = 0; i < n; i++)
    for (int c = S; c >= a[i]; c--)
        if (can[c - a[i]]) can[c] = true;
```

Coin change - unlimited copies

```
// MINIMUM number of coins to make S
vector<ll> dp(S + 1, INF); dp[0] = 0;
for (int c = 1; c <= S; c++)
    for (int coin : coins)
        if (coin <= c && dp[c - coin] + 1 < dp[c]) dp[c] = dp[c - coin] + 1;
// dp[S] == INF means impossible

// NUMBER OF WAYS, order does NOT matter (combinations)
vector<ll> ways(S + 1, 0); ways[0] = 1;
for (int coin : coins) // coin loop OUTSIDE
    for (int c = coin; c <= S; c++) // FORWARD
        ways[c] = (ways[c] + ways[c - coin]) % MOD;

// NUMBER OF WAYS, order DOES matter (permutations): swap the two loops.
```

Longest increasing subsequence, $O(n \log n)$

```
vector<ll> t; // t is NOT the LIS - only its LENGTH is correct
for (ll x : a) {
    auto it = lower_bound(all(t), x); // strictly increasing
    // use upper_bound instead for non-decreasing (equal values allowed)
    if (it == t.end()) t.push_back(x);
    else *it = x;
}
int lis = t.size();
```

Edit distance and LCS - the classic $O(n*m)$ grid

```
// EDIT DISTANCE: min operations to turn s into t
for (int i = 0; i <= n; i++) dp[i][0] = i;
for (int j = 0; j <= m; j++) dp[0][j] = j;
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        dp[i][j] = (s[i-1] == t[j-1])
            ? dp[i-1][j-1]
            : 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});

// LONGEST COMMON SUBSEQUENCE
dp[i][j] = (s[i-1] == t[j-1]) ? dp[i-1][j-1] + 1
    : max(dp[i-1][j], dp[i][j-1]);
```

Paths in a grid

```
// number of ways to walk from (0,0) to (n-1,m-1) moving right/down only
vector<vector<ll>> dp(n, vector<ll>(m, 0));
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++) {
        if (g[i][j] == '#') { dp[i][j] = 0; continue; }
        if (i == 0 && j == 0) { dp[i][j] = 1; continue; }
        if (i) dp[i][j] += dp[i-1][j];
        if (j) dp[i][j] += dp[i][j-1];
        dp[i][j] %= MOD;
    }
```

Linear DP on the array itself

Enormously common at 1300-1500. State = "best answer considering the first i elements, with element i in some status". Two typical shapes:

```
// max subarray sum (Kadane)
ll best = a[0], cur = a[0];
for (int i = 1; i < n; i++) {
    cur = max(a[i], cur + a[i]);
    best = max(best, cur);
}

// "cannot take two adjacent" (house robber)
// dp[i][0] = best for first i, i not taken; dp[i][1] = i taken
dp[0][0] = 0; dp[0][1] = a[0];
for (int i = 1; i < n; i++) {
    dp[i][0] = max(dp[i-1][0], dp[i-1][1]);
    dp[i][1] = dp[i-1][0] + a[i];
}
ll ans = max(dp[n-1][0], dp[n-1][1]);
```

9. STL and strings - stop wasting minutes on syntax

map, set, counting

```
map<ll,int> cnt;
for (ll x : a) cnt[x]++;           // frequency table
if (cnt.count(x)) ...              // careful: cnt[x] CREATES the key
for (auto [k, v] : cnt) ...        // iterates in sorted key order

set<int> s;
s.insert(x); s.erase(x); s.count(x);
auto it = s.lower_bound(x);        // smallest element >= x
if (it != s.end()) { int ge = *it; }
if (it != s.begin()) { --it; int lt = *it; } // largest element < x

multiset<int> ms;                   // allows duplicates
ms.erase(ms.find(x));              // erase ONE copy. ms.erase(x) kills ALL.
*ms.begin() // smallest
*ms.rbegin() // largest
```

Coordinate compression - big values, small n

```
vector<ll> v = a;
sort(all(v));
v.erase(unique(all(v)), v.end());
for (int i = 0; i < n; i++)
    a[i] = lower_bound(all(v), a[i]) - v.begin(); // now a[i] is in 0..v.size()-1
```

Strings

```
string s; cin >> s;                // reads one token, stops at whitespace
getline(cin, s);                   // whole line. After a cin>>x you MUST first
                                   // do cin.ignore(); or you read an empty line.
s.substr(i, len)                   // len characters starting at i
s.find("ab")                       // index, or string::npos if absent
count(all(s), 'a')
reverse(all(s));
sort(all(s));                      // anagram test: sort both, then compare
to_string(x);  stoll(s);  stoi(s);
s[i] - 'a'                          // 0..25 for lowercase
(char)('a' + k)                    // back to a character

// palindrome check
bool pal = true;
for (int i = 0, j = s.size() - 1; i < j; i++, j--)
    if (s[i] != s[j]) { pal = false; break; }
```

Small things that cost people real minutes

```
max({a, b, c}); min({a, b, c});    // note the braces for 3+ arguments
accumulate(all(a), 0LL);           // 0LL, not 0, or the SUM is an int
*max_element(all(a));               // value; drop the * for an iterator
max_element(all(a)) - a.begin();    // index of the maximum
abs(x)                             // works on long long in C++11 and later
swap(a, b);
vector<vector<ll>> dp(n, vector<ll>(m, 0)); // 2D vector, all zeros
fill(all(a), 0);
a.assign(n, 0);                    // resize AND reset in one call
```

10. Read this before every single submission

This page costs you forty seconds. A wrong submission costs you twenty penalty minutes and a chunk of your confidence. Read it every time.

The overflow check

- Can any sum or product in my code exceed 2,000,000,000? If yes, is EVERY variable in that chain a long long?
- `int a, b; ll c = a * b;` is STILL WRONG. The multiply happens in int. Write `(ll)a * b`.
- `1 << 40` is wrong. Write `1LL << 40`.
- `mid = (lo + hi) / 2` overflows when both are near $1e18$. Write `lo + (hi - lo) / 2`.
- lcm: `a / gcd * b`, never `a * b / gcd`.
- Am I adding to something already set to INF? If INF is $1e18$ and I add to it twice, it wraps.

The multi-test-case check

- Did I actually read `t`? Is my while (`t--`) there?
- Every global array, vector, adjacency list, visited array, map, set and answer variable: is it reset?
- Am I resetting them in $O(n)$, not in $O(BIG_N)$? Clearing a $1e6$ array $1e4$ times is $1e10$ operations.
- Did I leave a "return 0;" or a debug print inside `solve()`?

The edge-case check - run these in your head

Case	Does my code survive it?
<code>n = 1</code>	Loops that start at <code>i = 1</code> , or access <code>a[i-1]</code> , or assume a pair exists.
<code>n = 2</code>	Any logic that needs a "middle" element.
All elements equal	Anything comparing strictly (<code><</code>) instead of (<code><=</code>).
All elements distinct	Anything relying on duplicates existing.
The answer is 0, or is negative	Did I initialise <code>best = 0</code> when the answer can be negative? Use <code>-INF</code> .
Maximum values everywhere	This is the overflow case. <code>n = 2e5</code> and <code>a[i] = 1e9</code> gives a sum of $2e14$.
The array is already sorted / already correct	Does my "number of operations" answer come out as 0?
Empty result, no valid answer exists	Do I print <code>-1</code> exactly as the statement asks?

The output check

- "YES"/"NO" vs "Yes"/"No" vs "yes"/"no" - copy the exact spelling from the statement.
- Is there one answer per test case, on its own line?
- Did I re-read the output section of the statement AFTER writing the code? Do it now.
- If several answers are acceptable, mine may be valid even if it differs from the sample. Do not "fix" it to match.

11. Red verdict? Go here before touching the code

WRONG ANSWER

1. Re-read the statement. Out loud. This finds the bug more often than anything else on this list.
2. Check the test number. WA on test 1 means you misread the problem. WA on test 40 means an edge case or an overflow.
3. Run the overflow checklist from the previous page.
4. Run the edge case table: $n = 1$, all equal, maximum values.
5. 0-indexed vs 1-indexed. Did you mix them in the same array?
6. Did you reset everything between test cases?
7. Write a 5-line brute force and compare on small random inputs. If you have 30+ minutes left, this is faster than staring.
8. If you have re-read it twice and found nothing: your ALGORITHM is wrong, not your code. Stop debugging and rethink.

TIME LIMIT EXCEEDED

- Is `ios_base::sync_with_stdio(0); cin.tie(0);` actually in main?
- Are you using `endl` instead of `"\n"`? `endl` flushes on every call.
- Count the real complexity. Two nested loops over $n = 1e5$ is $1e10$, which is a hundred times too slow.
- Are you passing a vector to a function BY VALUE inside a loop? Use `const vector<ll>&`.
- Clearing a big global structure per test case? Size it to n instead.
- `map` and `set` are $O(\log n)$ with a heavy constant. Sorting a vector is usually faster.
- Are you building a string with `+=` inside a hot loop, or printing with `cout` inside a $1e6$ loop without `"\n"`?
- TLE on test 1 or 2 usually means an infinite loop, not slowness. Check your while conditions.

RUNTIME ERROR

- Array index out of bounds. Did you write `a[n]` somewhere, or index a 1-indexed array with 0?
- `adj[]` sized N but node numbers go to $n+1$? Size everything to $N = \max N + 5$.
- Division or modulo by zero.
- Recursion too deep - a DFS on a $1e5$ -node path will overflow the stack. Rewrite it as BFS.
- Did you reserve a vector but never resize it, then index it?

MEMORY LIMIT EXCEEDED

- A 2D array of $1e5 \times 1e5$ does not exist. Rethink the state.
- `int` is 4 bytes, `long long` is 8. A $1e7$ array of `long long` is 80 MB - usually over the limit.
- Not clearing vectors between test cases while pushing into them.

The debugging rule that saves contests

If you have been debugging alone for 10 minutes with no progress, you are now the worst person in the room to find this bug. Hand the printed code to a teammate and go read a different problem. Come back in 20 minutes. Never let one person occupy the keyboard for more than 15 minutes on a problem that is not converging.

12. Team protocol - where the extra solve comes from

One keyboard, three brains. Almost all of a weak team's lost points come from bad allocation, not bad algorithms. Agree to this before the contest starts, in writing, and follow it even when it feels wrong.

The timeline

Time	What happens
0:00 - 0:10	Split the problem set three ways. One reads the front, one the middle, one the back. NOBODY TOUCHES THE KEYBOARD. Each person writes a one-line summary and a difficulty guess for their problems.
0:10 - 0:15	Share all summaries out loud. Rank every problem: EASY / MAYBE / NO. Start coding the easiest one immediately.
0:15 onward	One person codes. The other two read and design on paper. The coder does not think about new problems; the thinkers do not touch the keyboard.
Scoreboard live	RE-SORT EVERYTHING BY SOLVE COUNT. The crowd is a better difficulty estimator than you are. If 40 teams solved H and 3 solved B, you were wrong about B.
Last 45 min	Stop starting new problems. Finish what is half-written. Submit anything that is close.
Last 10 min	Submit everything that compiles and passes the samples. A blind submission that works is a solve; an unsubmitted correct solution is nothing.

Hard rules

- 20-MINUTE RULE. No working idea after 20 minutes of thinking? The problem goes back on the pile and a different person reads it fresh. Fresh eyes on an unread problem beat tired eyes on a stuck one.
- NEVER TWO PEOPLE ON ONE PROBLEM unless it is a debugging emergency. You have three problems in flight at all times, not one.
- WRITE THE IDEA ON PAPER BEFORE TYPING. If you cannot write the algorithm in four lines on paper, you are not ready to code it, and you will waste 25 minutes discovering that.
- THE CODER ANNOUNCES what they are typing. "I am doing D, binary search on the answer, about ten minutes." The other two then know the keyboard schedule.
- SAMPLE TEST BEFORE SUBMITTING. Always. Including the sample you think is trivial.
- PRINT THE CODE when a debug drags past 10 minutes, and let the keyboard go to someone else.

First three minutes on any new problem

1. Read the CONSTRAINTS. n , t , the time limit. Write down the complexity you are allowed.
2. Read the OUTPUT section. What exactly are you being asked to print?
3. Read the SAMPLE EXPLANATION. Not the sample output - the explanation.
4. Now read the story. Restate it in one sentence to a teammate.
5. Only now open Section 2 of this sheet and look for a trigger.

13. Your pattern index - fill this in before the contest

This is the most valuable page in the sheet and it is currently mostly empty, because it has to be yours. Every problem you upsolve between now and the contest, add ONE line. Write the SIGNAL that gave it away on the left, and the IDEA on the right. Never the code.

Starter entries - recurring ideas at the 1300-1600 level

Signal in the statement	Idea
Only allowed to increase elements, want non-decreasing	Sweep left to right, $ans += \max(0, prev - cur)$, set $cur = \max(cur, prev)$.
Minimum moves to make all equal, move = ± 1 on one element	Sort, target is the median, answer = sum of $ a[i] - median $.
Choose a subset to maximise sum with a count limit k	Sort descending, take the first k. Prefix sums make all k answerable at once.
"Split the array into k parts, minimise the largest part"	Binary search on the largest part; check greedily by cutting whenever the running sum exceeds x.
Swap any two elements any number of times	You can reach any permutation. So the answer only depends on the MULTISSET of values. Sort and forget order.
Only adjacent swaps allowed, count minimum swaps	That is the number of inversions.
"Remove elements so no two adjacent are equal"	Greedy sweep, keep the first of each equal run.
Every element must be paired with another	Sort, then either pair neighbours (minimise) or pair ends (balance). Try both on the sample.
Answer over all subarrays, and elements are 0/1	Count runs of consecutive equal values. A run of length L gives $L*(L+1)/2$ subarrays.
Circular array	Duplicate it: $a = a + a$, then treat windows of length n on the doubled array.
Query: how many earlier elements are less than $a[i]$	Sweep and maintain a count array over compressed values (or a set, if you only need one).
Grid, must reach a target, some cells blocked	BFS. If there are costs 0 and 1 only, 0-1 BFS with a deque.
"You may perform the operation at most once"	Precompute prefix and suffix, then test all n positions. $O(n)$, not $O(n^2)$.
The value of the answer is tiny (0, 1, 2 or -1)	Case analysis, not an algorithm. Prove when each case happens on paper.
Two players, last to move wins, n up to $1e9$	Parity. Brute force $n = 1..10$ by hand, find the pattern, print a formula.

Last thing

This sheet does not make you a better competitive programmer. It makes you a faster, less error-prone version of the programmer you already are.

The gap between 3 solves and 4 solves is almost never a missing algorithm. It is twenty minutes lost to an int overflow, or forty minutes spent on the wrong problem because nobody checked the scoreboard.

Fix those two things and the fourth problem shows up on its own.

Your own entries - write these in pen as you practise

[illegible]

Your own entries (continued)

[illegible]